



CG2111A Engineering Principle and Practice
Semester 2 2024/2025

“Alex to the Rescue”
Final Report
Team: B01-5A

Name	Student #	Position	Main Role
Ayden Van Etten	A0302605U	Team Leader	TLS Client, Movement and Color Detection Programmer
Ang Wei Bin	A0301853L	Member	LiDAR, SLAM & Camera Programmer
Aw Shuo Jie	A0309254E	Member	MedPack Delivery & claw System designer
Royden Lim Yi Ren	A0301667E	Member	Hardware & Claw designer, Professional wire Manager & Trained Alex Operator
Josh Vasanth	A0271834B	Member	Spectator (Limited Involvement, occasionally present at meetings)

Table of Contents

Section 1: Introduction	2
Section 2: Review of State of the Art	3
Section 3: System Architecture	4
Section 4: Hardware Design	4
Section 5: Firmware Design	6
5.1 High Level Algorithm on the Arduino Mega	6
5.2 Arduino-Pi Communication Protocols	7
5.3 Movement Execution Protocol	8
5.4 Claw & Dispenser Control	9
Section 6: Software Design	9
6.1 High Level Algorithm on Pi	9
6.2 Teleoperation	9
6.3 Colour Detection	11
6.4 LiDAR Mapping	12
6.5 SLAM GUI Display	13
Section 7: Lessons Learnt	13
7.1 Most Important Lessons	13
7.2 Greatest Mistakes	14
Appendix A – Input Keys, IDs and Command Types	15
Appendix B – Photo Documentation of Alex	38
Appendix C	41
Appendix D	42
References	43

Section 1: Introduction

The “Moonbase CEG Rescue Mission” simulates a high-stakes emergency scenario, where a catastrophic explosion has endangered astronauts at a lunar base. In such environments where human intervention is implausible, robotic systems play a vital role in ensuring safety and survival. To address this challenge, our team developed Alex—a remotely operated Search and Rescue (SAR) Robot, designed to navigate the lunar terrain and save astronauts.

Alex must be capable of navigating through this simulated, obstacle filled, lunar environment while accurately mapping & navigating the area with a LIDAR-based system. Alex will encounter astronaut figures with two colours indicating their medical urgency: "Green" indicating the need for a Med Pack, and "Red" indicating serious injuries, necessitating the need to be physically relocated to a designated safe zone. To accomplish this, Alex will be teleoperated, with its operator relying solely on real-time data and maps, without direct visual assistance, emphasizing the importance of precise sensor integration and control algorithms.

The rescue problem also introduces operational constraints, including limited camera use and time constraints, challenging us to innovate and design robust hardware-software systems. Ultimately, Alex serves as a prototype for real-world robotic responders that can assist in disaster zones, space missions, and hazardous environments where human life is at risk .

Section 2: Review of State of the Art

i. “Earthshaker” – Winning Teleoperated Rescue Robot (USTC, A-TEC international)

Earthshaker is a teleoperated SAR robot. It features a tracked chassis for stable terrain traversal and a swing-arm with a dozer-blade to clear debris and climb stairs. For object manipulation it has a robotic arm with a two-finger gripper that enables fine object manipulation such as pressing buttons.

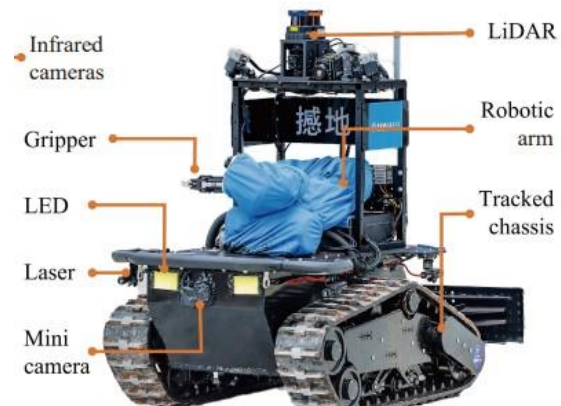


Figure 1: EARTHSHAKER

It has six cameras (wide-angled & infrared), a 2D LIDAR, and an IMU for environmental awareness and mapping. The low-level motion and peripherals are managed by an STM32 microcontroller, while a NUC minicomputer executes high-level mapping and semi-autonomous control. 2x AT95 remote controllers are used to operate its motion and arm.

(Zhou et al., 2022)

Strengths: Terrain adaptability, Many Sensor Capabilities, and semi-autonomous assistance.

Weaknesses: Large size (250 kg, 1.3 m width), limits deployment in confined spaces.

ii. “CENTAURO”– Hybrid Mobility Disaster Robot (IIT, European Union)

CENTAURO is a semi-autonomous disaster-response robot equipped with hybrid locomotion—four articulated 5-DOF legs with steerable wheels allow both fast wheeled motion and precise stepping over debris. CENTAURO has a humanoid upper body with two 7-DOF robotic arms—one equipped with a 9-DOF Schunk hand for precise manipulation tasks like drilling, the other with a 1-DOF SoftHand for general tasks. (Klamt et al., 2018).

It has an array of sensors which includes: a Kinect RGB-D sensor, 3 wide angled head cameras, a rotating 3D laser scanner, foot cameras & a torso-mounted IMU (used for balance and motion tracking). The system runs on a ROS-based architecture across three CPUs.

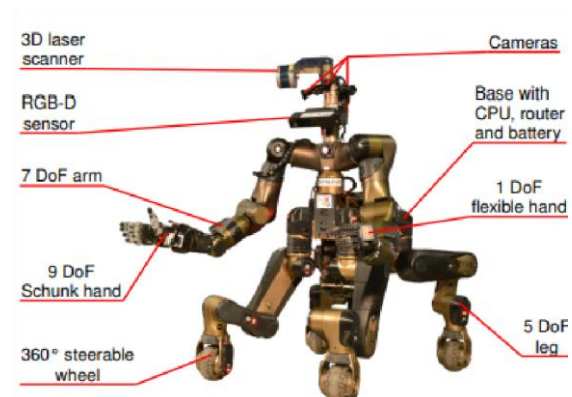


Figure 2: CENTAURO

Teleoperation is done via multiple interfaces: a Logitech joystick for omni directional driving, a 6D SpacePilot for precise wrist control, and a GUI for semi-autonomous stepping motions.

Strengths: Exceptional mobility in unstructured environments, advanced manipulation
Weaknesses: Complex design and controls and bulky size hinder rapid deployment and navigation in confined areas. (Klamt et al., 2018)

Section 3: System Architecture

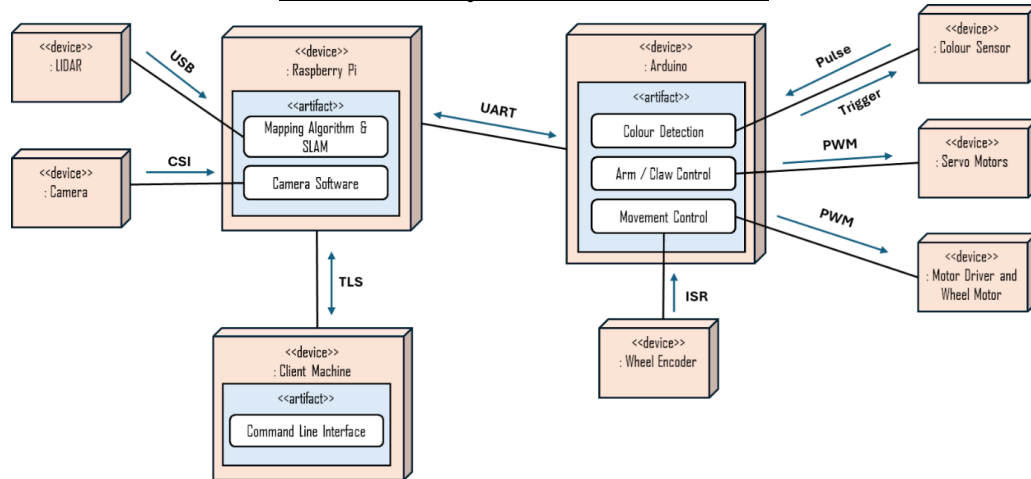


Figure 3: System Architecture, the upscaled image can be found in Appendix C

Section 4: Hardware Design

Design Philosophy

Although cable management was not a formal requirement, our team prioritised maintaining a compact and tidy design for several reasons. A well-organised layout facilitates easier hardware debugging, reduces the risk of entanglement during movement, and contributes to the robot's visual appeal. Additionally, it minimises risk of knocking over obstacles during operation. Beyond layout efficiency, physical dimensions and weight distribution were also key considerations. The total height of Alex was deliberately kept below 20cm to ensure our LiDAR could detect all obstacles, which are minimally 21cm tall. Furthermore, heavier components were positioned closer to Alex's base, lowering his centre of gravity and increasing his stability.

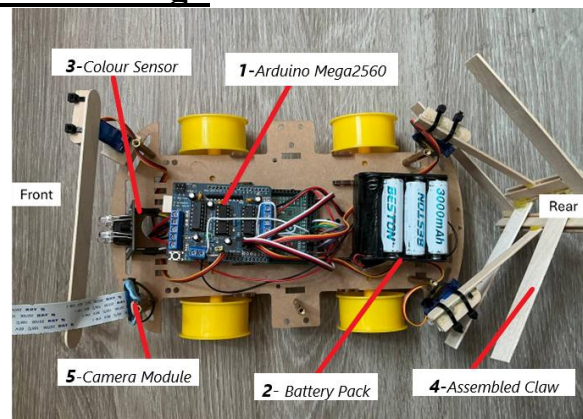


Figure 4

Base Layer

The bottom layer houses 4x DC motor coupled wheels, mounted to the chassis. 2x Rotary encoders are installed at each of the front wheels to track its rotation.

1. Arduino Mega2560: USB port facing _____ outward for easy access and testing.

2. Battery Pack: Mounted close to the motor driver to minimise wire length.
3. Colour Sensor: Positioned close to Arduino to minimise wire complexity.
4. Claw Assembly: Rear-mounted to avoid obstruction of colour sensor.
5. Camera Module: Location chosen, to match cable length constraints and top-frame layout.

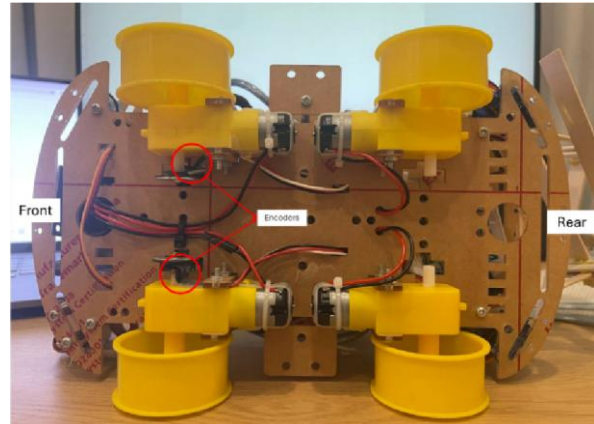


Figure 5

Top Layer

- a. Medpack Dispenser: Mounted at the front for convenient deployment once
2. Power Supply: Positioned Centrally to elevate LiDAR & for simple charging.
3. LIDAR: Elevated for unobstructed scans. 360°
4. Raspberry Pi: Placed for easy access and short, strain-free wiring to key components.



Figure 6

Use of Motor Driver Header Pins

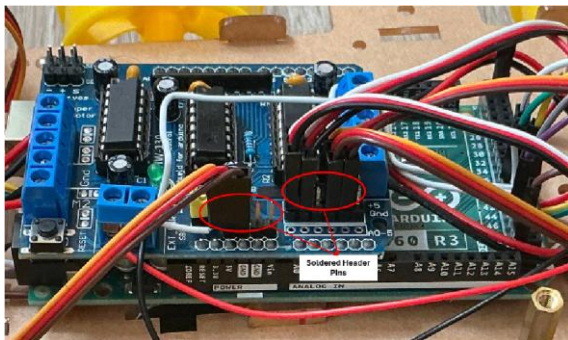


Figure 7

During the initial assembly phase, a breadboard was used to compensate for the limited 5V and GND outputs available on the Arduino.

However, we later identified that the L293D Motor Shield has multiple unused 5V and GND terminals. To optimize space and reduce wiring complexity, male header pins were soldered directly onto the motor shield. This enabled

direct connection of servos and encoders, eliminating the need for a breadboard. This modification significantly reduced wire clutter and contributed to a cleaner, more organised hardware layout, while also improving electrical reliability.

Cable Management & Wiring

To maintain a compact layout and ensure safe operation, wiring across all components was carefully routed and optimized. Excess wire was minimized, and flexible, low-profile materials were used where possible to improve reliability and accessibility.

Component	Connections	Considerations and Modifications
Colour Sensor	6 wires	Wires were routed beneath the Arduino and guided to the opposite end to minimize clutter and avoid interference with components or moving parts.

Encoders and Servo Wiring	Vcc, GND, and signal	Encoders and servos were connected directly to header pins soldered onto the motor driver. Original jumper wires were replaced with softer, white wires for better flexibility and space efficiency. Each wire was trimmed to exact lengths and neatly routed around solder joints to improve cable management and accessibility.
Wheel Motors	Motor +/-	Fixed-length cables were bundled and secured under the chassis using zip ties to prevent tangling and ensure clearance from the wheels during operation.

Power Supply Modification

The provided 8-cell battery pack could only fit 7 batteries due to design limitations. To supply more voltage, 2x 4-cell holders were soldered together in series, creating a compact and efficient 8-cell power source. This optimises motor performance and torque output.

Claw Assembly

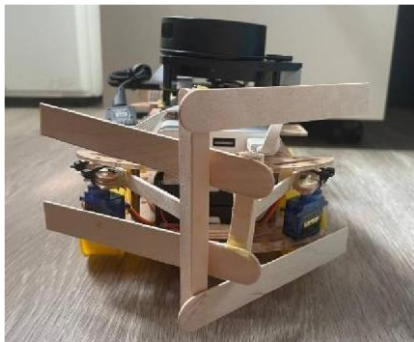


Figure 8

The claw was constructed using ice-cream sticks and hot glue, chosen for their light weight and sufficient structural integrity. Each segment was connected to an individual servo motor to optimize control while minimizing overall mass. The claw's caged design allowed it to securely envelop the astronaut figure, reducing the likelihood of tipping. Additionally, the claw's ability to open widely increased the angular range at which it could effectively grasp the astronaut.

Medpak Dispenser

The Medpak dispenser was designed with simplicity and functionality in mind. It consisted of a gentle ramp and a movable arm, both made from ice-cream sticks and hot glue. The arm was actuated by a servo motor and operated via a single command. Upon activation, the arm opens, allowing the Medpak to roll down the ramp and onto the ground toward the astronaut.

Section 5: Firmware Design

5.1 High Level Algorithm on the Arduino Mega

Our Arduino Mega, first setups the relevant interrupts and the DDR of pins, subsequently initializing serial communication and polling for incoming packets in its main loop().

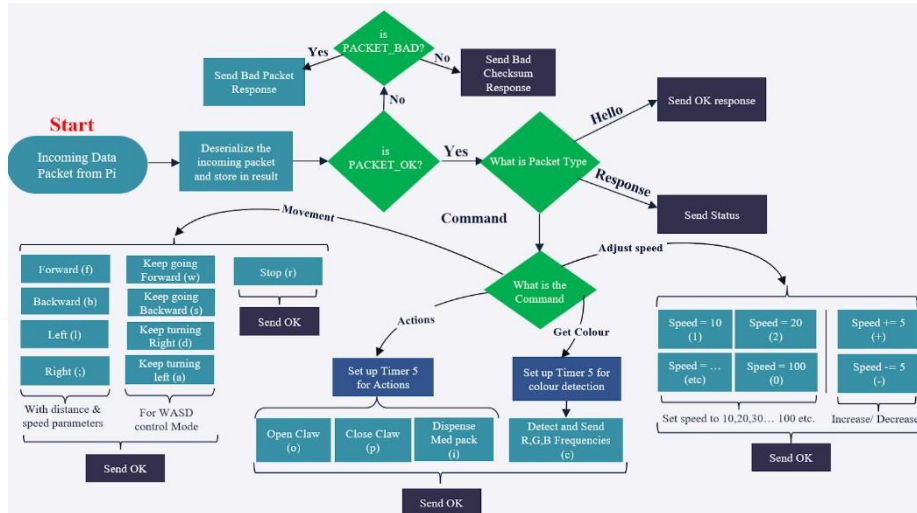


Figure 9: High level Algorithms and Commands within our Arduino Mega, A higher resolution version of this can be found in Appendix D.

5.2 Arduino-Pi Communication Protocols

Communication between the Arduino and the Raspberry Pi occurs over asynchronous UART over a USB serial connection, configured with an 8N1 format at 9600 baud. Data is serialized by the Pi into a fixed-length communication packet called TComms, which consists of:

Magic Number	Data Size	Payload Data (Buffer)	Checksum	Padding
4 bytes (0xFCFDFEFF)	4 bytes	128 bytes	1 byte	3 bytes

The Arduino deserializes incoming packets, verifies the Magic Number, and checks the checksum to ensure data integrity. If the packet is valid, it extracts the core data structure, TPacket (100 bytes), which contains the command and associated data. Its structure:

Packet Type	Command or Response Type	Padding	Payload Data	Parameters
1 byte	1 byte	3 bytes	32 bytes	64 bytes

Once deserialization is complete and the command is executed, the Arduino similarly constructs an OK response as a TPacket to signal the completion of a command. This response is then also serialized into a TComms structure and transmitted back to the Pi to confirm successful execution. The identifier code for the various Command and Response types are included in Appendix A, Table 1,2 & 3.

Arduino-Pi Handshake

Upon startup, the Arduino enters a wait state, continuously polling for a PACKET_TYPE_HELLO from the Raspberry Pi to initiate the handshake sequence. The Pi sends this packet immediately after establishing the serial connection. Once the Arduino receives and validates it, it responds with an OK packet, signaling that communication has been successfully established (Appendix A, Listing A1).

Communication Limitations and Timing Considerations

The current implementation does not feature automatic synchronization between command issuance and response handling. Specifically, the Pi does not wait for an OK response before it is able to send the next packet. This design choice was made to keep communication logic simple and to reduce software complexity. Instead, this is done manually: the operator visually confirms the receipt of an OK response, displayed on the client interface, before sending the next command. While this necessitates the operator's attentiveness, it helps prevent buffer overflows and desynchronization issues that we had previously encountered. During testing, we observed that sending packets in rapid succession led to checksum mismatches. This was due to the Arduino partially reading a new incoming packet while still processing the previous one, leading to desynchronization and failure to detect the correct Magic Number, likely due to the UART buffer overlapping between packets. To quantify timing constraints:

Transmission Time per byte: (8N1 Format, 9600 baud)

Total Estimated Transmission Time:
$$14510140 \div 9600 \times 8 \times 1.20417 \approx 300 \approx 1.45 \text{ ms}$$

 0417ms 145 .ms8ms (140-(receivebyte and packet) respond)

Receiving a Round-trip full command Time packet = takes approximately 146ms. However, the Arduino only begins polling for the next packet after processing the current command and sending an OK response. While the UART connection is technically duplex, our implementation is effectively half-duplex—The Arduino does not begin polling for the next packet until it has fully processed the current one and sent a response. As such, the full round-trip time (~300ms) provides a more realistic estimate for safe command spacing. This delay threshold is used throughout the system and discussed further in later sections.

5.3 Movement Execution Protocol

Bare Metal Programming of the Motors

As a challenge, we had implemented bare-metal programmed motor control (*Appendix A, Listing A2*), steering away from the recommended Adafruit Motor Library. Despite this having minimal functional benefits, we did so to test our understanding of the concepts learnt in EPP2. Though, the main challenge of this was interpreting the L293D and Atmega2560 datasheets (Texas Instruments, 2016) (Microchip Technology Inc., 2014).

General Movement

To control Alex's movement, we implemented two distinct systems: one based on the movement logic from Studios 13 & 14 (normal mode), and the other built around a custom algorithm using the ncurses library (ncurses mode). While normal mode parses distance and speed parameters for discrete movements (see *Appendix A, Listing A3*), ncurses mode continuously moves Alex in a given direction until a new command is received. As normal mode was seldom used, this report will focus primarily on ncurses mode.

5.4 Claw & Dispenser Control

The claw and dispenser mechanisms are actuated by 3 servo motors connected to Timer5 via output compare channels: OC5A (right claw), OC5B (left claw), and OC5C (dispenser gate). Timer 5 is configured in Fast PWM Mode 14, which provides precise pulse-width modulation for stable servo control.

Initialization is handled by the `setup_claw()` function, which disables interrupts, resets Timer 5 registers, and configures pins 44–46 as outputs. A 20ms PWM period is achieved by setting the Input Capture Register (ICR5) to 40,000, with a prescaler of 8 and a 16 MHz clock. Initial pulse widths are set to close all three mechanisms (*Appendix A, Listing A12*).

To enable smooth, coordinated movement of the claw, the `smoove()` function incrementally adjusts PWM values in two stages. It first decreases the right claw's position (OCR5A) to a defined interception point, then increases the left claw's position (OCR5B) to the target. This staged motion prevents sudden jerks and creates a fluid, natural gripping action—especially useful for handling delicate objects such as astronaut figures. The `close_claw()` function uses `smoove()` for controlled closure, while `open_claw()` first opens the left claw (OCR5B = 1000), applies a short delay, and then extends the right claw outward (OCR5A = 5000). These actions enable safe object interaction during rescue tasks (*Appendix A, Listing A4*).

Medpack Dispenser Control

The medpack dispenser operates using a third servo connected to OCR5C. The `dispense_medpack()` function triggers the dispenser mechanism by briefly setting the PWM value to 1000, which opens the gate. After a 500ms delay allowing the medpack to drop onto the maze floor near the astronaut, the PWM value is reset to 4000 to close the gate. This mechanism ensures reliable and efficient delivery of the medpack. The action is timed to ensure consistency and avoid accidental re-triggering due to mechanical lag. (*Appendix A, Listing A5*.)

Section 6: Software Design

6.1 High Level Algorithm on Pi

1. Initialize TLS Communication between Raspberry Pi and Teleoperation Client
2. Begin LiDAR data Acquisition, while filtering data based on quality, angle and range, and publish to lidar scan Topic in Pub/Sub
3. Generate Map using BreezySLAM based on the filtered LiDAR scans.
4. Publish real-time LiDAR scans and SLAM-generated maps to the visualization module via Bokeh, using websockets.
5. Parse and convert teleoperation commands sent from the client into TPacket format, then wrap this into TComms packets to be transmitted to the Arduino via serial communication.
6. Repeat Steps 2–6 until Alex completes his mission (or if time runs out)

6.2 Teleoperation

Secure Communication Protocol

To ensure secure remote control, the Raspberry Pi initializes a TLS server during startup using a signed certificate and private key. Only clients with certificates signed by the same Certificate Authority (CA) are allowed to connect. Once the TLS handshake is completed, an encrypted communication channel is established between the client and the Pi.

Client Software

Alex is teleoperated through a C++ TLS client which launches two concurrent threads upon establishing communication (*Appendix A, Listing A6*):

- Reader Thread: Continuously polls for responses from the Pi..
- Writer Thread: Continuously polls for user input and sends corresponding commands.

The writer thread serializes each command into a fixed 10-byte packet, which is encrypted and transmitted over the TLS channel. The packet format is as follows:

Packet Type	Command ID	Parameter 1 (Dist/Angle)	Parameter 2 (Speed)
1 byte	1 byte	4 bytes	4 bytes

Movement and Implementation of ncurses

The writer thread also has a noteworthy implementation of the software library *ncurses*. By pressing 'm', the operator enters ncurses mode, where keys are polled continuously using getch() without needing *Enter* to be pressed. If the user is not pressing anything, getch() will return ERR, and a stop command would be sent. If the user pressed 'w', 'a', 's' or 'd', a corresponding movement command would be sent and a stop command would only be sent once the operator lets go of the key, when getch() starts returning ERR again (*Appendix A, Table 3*).

To prevent spamming of the Pi and Arduino as a result, our algorithm tracks the last command sent and will not send repeated commands, only sending new ones (*Note 1 in Appendix Listing A6-b*). Additionally, to prevent the client itself from being flooded with user inputs, flushinp() is called on each loop iteration (*Note 2 in Appendix Listing A6-b*). To exit ncurses mode, the operator can press 'q' where endwin() will deallocate memory from the window and subsequently return the client to 'normal mode'.

This design allows Alex to be steered with the same fluidity as a game character, enabling precise real-time control without needing predefined distances or Enter key presses. Although it lacks distance tracking, it alleviates small delays in manual inputs. Preventing the repeated entering of additional values and the confirming of commands which would otherwise compound over time; ultimately resulting in significant time savings.

Timing Constraints and Delay Strategy

Due to desynchronization issues in serial communication, observed during testing, a safe delay between command inputs was implemented. Within the *ncurses* loop, two delays are applied:

1. timeout(300), Waits up to 300ms for a keypress before getch() returns ERR.
2. napms(150), Adds a 150ms pause between loop iterations.

This timing buffer is **150ms longer than the previously calculated safe delay of 300ms**, as that was the derived MINIMUM interval required to avoid desynchronization and checksum errors. This additional margin accounts for real-world variability, such as interrupt delays, software stalls or minor processing overhead on the Arduino.

One side effect of this strategy is that Alex will always continue moving for at least 450 ms after any directional command is issued. To maintain precise control despite this fixed movement window, we implemented a dynamic speed adjustment system within ncurses mode. By pressing number keys (0–9), the operator can modify Alex's speed on the fly (*Appendix A, Table 3*). Since $\text{distance} = \text{speed} \times \text{time}$, this enables us to appropriately compensate for the delay and retain precise control over short-distance movements.

This approach proved effective: during testing, the system maintained reliable command delivery and prevented packet corruption or checksum mismatches. It also allowed for precise, responsive teleoperation without compromising communication stability.

6.3 Colour Detection

Colour detection is initiated by the teleoperator entering 'c' in the client interface, prompting the Raspberry Pi to send a command to the Arduino. (*Appendix A, Table 3*).

As Timer 5 is already used for claw and dispenser control, its reconfiguration for colour detection is needed. Motor control relies on other timers, making Timer 5 the most viable shared resource. The set up for TCCR5B follows (*more details in Appendix A, Listing 10*)

```
TCCR5B = (1 << ICNC5) | (1 << ICES5) | (1 << CS50);
```

This sets Timer 5 to capture on rising edge with no prescaler, while enabling the noise canceller to enhance the reliability of our readings. Information about an object's colour is obtained by measuring the frequency of the output pulses from the colour sensor. The sensor emits pulses corresponding to the intensity of light detected by its red, green, and blue photodiodes. By enabling each colour filter sequentially and measuring the resulting pulse frequency, the colour of the object can be derived.

Our software algorithm to measure the frequency of pulses emitted is depicted below and in *Appendix A listing 11*:

1. Enables 100% frequency scaling (S1 & S0 = HIGH), suited for high-resolution measurement at 16 MHz with no prescaler.
2. Enable Timer 5 Input Capture interrupt (ICIE5) and reset the total period count
3. Poll for completion of the TIMER5_CAPT_vect to compute the total period count of 11 full periods.
 - a. On the first rising edge reset TCNT5 = 0 (signalling start of the 1st period)
 - b. From the 3rd edge onwards, record and increment period by ICR5, reset TCNT5 after
 - c. After the 12th edge, set a done flag to signal completion of 11 full periods.

4. Determine average count for 1 period: $f_{detected} T_{avg} = T f_{osc} T_{11total}$, where f_{osc} is 16 MHz total period count

5. Derive the average frequency: where

6. Format results into a string for it to be sent to the teleoperator using `sendMessage()`

These steps are then done for the red (S1 & S2 LOW), green (S2 & S3 HIGH) and blue (S2 LOW & S3 HIGH) photodiodes (Texas Advanced Optoelectronic Solutions, Inc., 2009). Where the teleoperator can subsequently analyse the 3 detected frequencies for each colour channel to conclude on the object's colour.

While relying on human judgment may suggest a risk of human error, this approach overall enhanced our colour detection system. The red photodiode consistently read higher frequencies, and our readers were sensitive to environmental factors such as lighting conditions, background colour, and object distance. These variables are best accounted for by our trained operator, which with his extensive testing and experience, could reliably interpret frequency data and make accurate colour classifications.

6.4 LiDAR Mapping

The LiDAR A1M8-R1 was used to scan Alex's surroundings, operating in Express Mode with a sampling rate of up to 4000 samples/s. Despite the high throughput, the system maintained low bandwidth usage by compressing redundant data. To improve scan reliability, filtering was applied immediately upon data acquisition based on signal quality and distance. Samples with a quality score of less than 100 were discarded, as they typically corresponded to out-of-range readings and weak reflected laser pulses. Additionally, distances falling outside the LiDAR's effective range of 0.15–6m (Shanghai Slamtec Co., Ltd., 2020) were excluded. This preprocessing step ensured only relevant data was published to the lidar Scan topic, reducing noise and improving downstream map and display accuracy.

Simultaneous Localization and Mapping (SLAM)

BreezySLAM was selected for its ability to perform SLAM without odometry, making it a good fit for the Raspberry Pi (Levy, n.d.). It estimates the robot's position by aligning consecutive LiDAR scans, avoiding dependence on encoders, which can introduce errors due to wheel slip. This method, however, has limitations. Since all motion estimates come from aligning scans, small errors can build up over time. These alignment errors can distort the map and reduce positional accuracy, especially in tight spaces where precise movement is needed. To work around this, the operator relies on the raw LiDAR scan for accurate positioning, using the SLAM-generated map only as a rough reference. This approach allows the robot to navigate reliably while minimizing the effects of drift in the SLAM system.

6.5 SLAM GUI Display

The mapping utilizes Matplotlib for base map rendering and Bokeh for interactive web-based visualization. To ensure a fast real-time performance with minimum workload, the number of plots must be reduced. Since the LiDAR emits multiple laser readings at identical angles, we consolidate these by computing the mean distance from only high-quality readings at each angular position. The scans are resampled into 1° angular bins, a maximum total of 360. This data compression approach maintains spatial accuracy while dramatically decreasing processing overhead.

Section 7: Lessons Learnt

7.1 Most Important Lessons

Working Together as a Team

Throughout this project, we delegated roles across hardware, software, and firmware, allowing us to work in parallel and significantly improve team efficiency. The real challenge, however, came when integrating our individual components and ideas. This process revealed several issues, such as compatibility conflicts, which highlighted the importance of setting clear boundaries and constraints — such as physical space constraints, materials, consistent naming conventions and readable coding syntax. Establishing these parameters early on will make integration smoother and reduce potential conflicts.

We also learned to leverage each team member's strengths by assigning roles that best matched our individual skill sets. This not only helped us develop better solutions but also fostered more effective collaboration. We carried this lesson into the demonstration, where we divided responsibilities such as robot control, driver guidance, and map drawing.

Ultimately, this project taught us the essential skills for the professional world — teamwork and collaboration, where projects are rarely completed in isolation. Understanding each other's strengths and working styles was a key soft skill we developed and will carry forward.

Error Handling and the Problem-Solving Process

A significant lesson from this course centers on the critical interplay between robust error handling, iterative development, and practical problem-solving. Implementing new project functionalities and ideas revealed numerous potential points of failure alongside it; demonstrating the necessity of effective debugging skills.

This was especially evident during the integration of the ncurses library. Persistent issues, such as bad checksum errors, forced us to move beyond surface-level fixes and explore the deeper theoretical concepts behind serial communication. By analyzing transmission timing and communication flow, we were able to identify root causes that weren't immediately obvious, highlighting the value of connecting theory with real-world application.

When our initial error-handling strategies failed to fully resolve the bugs, iterative testing became essential. For example, to manage the flood of client-side inputs in ncurses mode, we experimented with several built-in functions like `timeout()`. However, through extensive testing, we found that repeatedly calling `flushinp()` provided the most consistent and reliable solution.

This experience emphasized that error handling isn't just about theoretical understanding—it also requires hands-on prototyping and iterative refinement to arrive at an effective solution.

7.2 Greatest Mistakes

Lack of Fine Movement Control

During our final run, we faced the greatest flaw in our system: the inability to precisely control Alex. Despite our repeated efforts to do the opposite of this, many external factors which were difficult to account for had either clouded or reduced the accuracy of our judgment as a result. This greatly affected Alex's ability to turn or move small distances precisely, in turn affecting our overall performance. These issues stemmed from three main factors:

1. **Significant Delay Time:** In a moment of panic just two hours before the final demo, we increased our `napms()` delay to 300ms in response to a sudden spike in checksum errors. This led to a total command delay of 600 ms—significantly higher than our previously consistent and well-tested 450 ms setup. Our experience had shown that increasing this delay reduced the odds of such errors; hence, it seemed like the 'safe call' at that time. However, this led us to lack precise control over Alex's movement in our final run despite our extensive past experience doing otherwise.

In hindsight, the cause of the checksum errors may not have been the delay itself, but external conditions: a critically low power bank, proximity to a Wi-Fi router, and the presence of other similarly-teleoperated systems in the room. These variables likely contributed to desynchronization more than the delay did, as our consistency with a 450ms delay supports so.

Nevertheless, we will learn to employ better debugging techniques in the future, to better quantify issues, and our instinctive solution to this: Revert back to a 450ms delay and employ measures for our Arduino to resynchronise and deal with a bad checksum error, by finding the next 'good' magic number and realigning itself.

2. **Slippery Floor Surface:** The robot's wheels frequently slipped due to inertia generated during rotation. While increasing friction could help, the robot's limited power constrained its ability to spin effectively under high friction.

Solution: Use a separate power supply capable of delivering sufficient power to the motors, enabling them to overcome friction and spin the wheels reliably.

3. **Inconsistent Motor Power:** The power supplied to the motors fluctuated depending on the battery's charge level. As a result, even with a fixed time command, the wheels rotated inconsistently.

Solution: Implement wheel encoders and interrupts to accurately measure wheel rotations and distance traveled, ensuring consistent movement regardless of power fluctuations.

SLAM Mapping Not Offloaded to Client Device

Another issue we encountered was delayed maze mapping due to the limitations of running SLAM locally on the Raspberry Pi. Since we relied solely on LiDAR-based mapping for navigation, the accuracy and responsiveness of SLAM were critical. Although we successfully integrated a lightweight SLAM system, its response time was noticeably laggy. This delay in

position estimation led to less efficient control of Alex, especially during time-sensitive decisions.

While not as critical as some of our other challenges, improving our SLAM system would be a high-impact enhancement. Offloading SLAM processing to a more powerful client device would significantly reduce the computational load on the Raspberry Pi, resulting in faster data processing and more responsive navigation.

Solution: Offload SLAM to a client machine. The Pi should transmit raw LiDAR data to the client, which can handle the computationally intensive SLAM operations. This would enable more accurate and real-time mapping, ultimately leading to smoother and more efficient robot control.

This project didn't just teach us how to build a robot. It taught us how to debug, collaborate and adapt under pressure to meet deadlines. These lessons extend far beyond the lab.

Appendix A – Input Keys, IDs and Command Types

Appendix A, Listing A1: Handshake with Pi

```
void wait_for_hello() {
    int exit = 0;
    while (!exit) {
        TPacket hello;
        TResult result;        do {            result =
        readPacket(&hello);    } while (result ==
        PACKET_INCOMPLETE);    if (result == PACKET_OK)
        {
            if (hello.packetType == PACKET_TYPE_HELLO)
            {
                sendOK();        exit = 1;            } else
            sendBadResponse();    } else if (result ==
            PACKET_BAD) {        sendBadPacket();            } else if
            (result == PACKET_CHECKSUM_BAD)
            sendBadChecksum();    }
    }
```

Appendix A, Listing A2: BARE METAL IMPLEMENTATION OF MOTORS

Listing A2-a: Registers Setup

```
void setupPWManDIR() {  
  // Set DDR  
  DDRE |= (1 << PE5); // FRONT_RIGHT  
  DDRE |= (1 << PE3); // BACK_LEFT  
  DDRB |= (1 << PB5); // BACK_RIGHT  
  DDRH |= (1 << PH3); // FRONT_LEFT  
  DDRD |= (1 << PD6);  
  DDRB |= (1 << PB0) | (1 << PB4);  
  
  DDRC |= (1 << PC7);  
  
  // Setup PWM  
  TCCR2A = (1 << COM2B1) | (1 << WGM20);  
  // Fast PWM, non-inverting  
  TCCR2B = (1 << CS21); // Prescaler 8  
  // Timer1  
  TCCR1A = (1 << COM1A1) | (1 << WGM10);  
  TCCR1B = (1 << WGM12) | (1 << CS11);  
  // Timer3  
  TCCR3A = (1 << COM3A1) | (1 << WGM30);  
  TCCR3B = (1 << WGM32) | (1 << CS31);  
  // Timer4  
  TCCR4A = (1 << COM4A1) | (1 << WGM40);  
  TCCR4B = (1 << WGM42) | (1 << CS41);  
}
```

Listing A2-b: Set Speed

```

void setSpeed(int motor, int speed) {
  if (motor == FRONT_LEFT) {      OCR4A
    = speed; // D6
  } else if (motor == FRONT_RIGHT) {
    OCR2B = speed; // D3
  } else if (motor == BACK_LEFT) {
    OCR3A = speed; // D5
  } else if (motor == BACK_RIGHT) {
    OCR1A = speed; // D11
  }
}

```

Listing A2-c: Movement Control

```

void move(float speed, int direction) {
  int speed_scaled = (speed / 100.0) * 255;
  // Set all motor speeds
  setSpeed(FRONT_LEFT, speed_scaled);
  setSpeed(FRONT_RIGHT, speed_scaled);
  setSpeed(BACK_LEFT, speed_scaled);
  setSpeed(BACK_RIGHT, speed_scaled);
}

```

```

    switch (direction) {
case GO:
    setDirection(FRONT_LEFT, 0);
setDirection(FRONT_RIGHT, 0);
setDirection(BACK_LEFT, 1);
setDirection(BACK_RIGHT, 1);
break;    case BACK:
    setDirection(FRONT_LEFT, 1);
setDirection(FRONT_RIGHT, 1);
setDirection(BACK_LEFT, 0);
setDirection(BACK_RIGHT, 0);
break;    case CW:
    setDirection(FRONT_LEFT, 1);
setDirection(FRONT_RIGHT, 0);
setDirection(BACK_LEFT, 0);
setDirection(BACK_RIGHT, 1);
break;    case CCW:
    setDirection(FRONT_LEFT, 0);
setDirection(FRONT_RIGHT, 1);
setDirection(BACK_LEFT, 1);
setDirection(BACK_RIGHT, 0);
break;    default:    //
case STOP:
    setSpeed(FRONT_LEFT, 0);
setSpeed(FRONT_RIGHT, 0);
setSpeed(BACK_LEFT, 0);
setSpeed(BACK_RIGHT, 0);    }
}

```

Appendix A, Listing A3: Basic Commands in ‘Normal’ Mode

Listing A3-a: Basic Commands in ‘Normal’ Mode

```

void forward(float dist, float speed)

```

```

{   if(dist > 0)      deltaDist =
dist;   else      deltaDist = 9999999;
newDist = forwardDist + deltaDist;
dir = (TDirection) FORWARD;
move(speed, GO);
}

```

Listing A3-b: Command for tracking turns in 'Normal Mode'

```

void left(float ang, float speed) {   if (ang == 0)
deltaTicks = 99999999;   else      deltaTicks =
computeDeltaTicks(ang);   targetTicks =
leftReverseTicksTurns + deltaTicks;   ccw(ang,
speed);
}

```

Listing A3-c: Void loop() logic for movement with distance tracking.

```

    if (dir == FORWARD) {      if
(forwardDist > newDist) {
deltaDist = 0;          newDist =
0;          stop();      }
    } else if (dir == BACKWARD) {

```

```

        if (reverseDist > newDist) {          deltaDist = 0;          newDist
= 0;          stop();          }      } else {          deltaDist = 0;
newDist = 0;          stop();          }      } if (deltaTicks > 0) {      if (dir
== LEFT && leftReverseTicksTurns >= targetTicks) {          deltaTicks = 0;
targetTicks = 0;          stop();          } else if (dir == RIGHT &&
rightReverseTicksTurns >= targetTicks) {          deltaTicks = 0;
targetTicks = 0;          stop();          } else {          deltaTicks = 0;
targetTicks = 0;          stop();          }
    }

```

Appendix A, Listing A4: (The complete implementation of `smoove()`, `close_claw()`, and `openclaw()`).

Appendix A, Listing A4-a: `smoove()`

```

void smoove (volatile uint16_t &ocrA, uint16_t targetA, volatile
uint16_t &ocrB, uint16_t targetB, uint16_t intercept, uint8_t step,
uint16_t delayTime){    while (ocrB <= targetB){        if (ocrA >=
targetA){            ocrA -= step;        }        if (ocrA <= intercept){
ocrB += step;        }        delay(delayTime);    }
}

```

Appendix A, Listing A4-b: open & close Claw

<pre> void close_claw() { smoove(OCR5A, 2500, OCR5B, 3300, 3700, 10, 5); } </pre>	<pre> void open_claw(){ // OCR5B = 1000; delay(1000); OCR5A = 5000; } </pre>
---	---

Appendix A, Listing A5: dispense_medpack()

```

void dispense_medpack(){
OCR5C = 1000;
delay(2000);    OCR5C =
4000;
}

```

Appendix A, Listing A6: Reader and Writer Thread

Appendix A, Listing A6-a: Reader Thread

```
void *readerThread(void *conn) {    char
buffer[128];    int len;    while (networkActive)
{        len = sslRead(conn, buffer,
sizeof(buffer));        printf("read %d bytes from
server.\n", len);        networkActive = (len >
0);        if (networkActive)
handleNetwork(buffer, len);    }
printf("Exiting network listener thread\n");
stopClient();
EXIT_THREAD(conn);
return NULL;
}
```

Appendix A, Listing A6-b: Writer Thread

```
void *writerThread(void *conn) {    int quit = 0;    while (!quit) {        char
ch;        printf("NORMAL MODE: f, b, l, k(right), STOP IS 'r' || OTHER MODE: m
"
        "(WASD) || GET COLOUR = c, OPEN CLAW = o, CLOSE CLAW = p,
MEDPACK
= "
        "i, h=clear stats, g=get stats
q=exit)\n");        scanf("%c", &ch);        // Purge
extraneous characters from input stream
flushInput();        char buffer[10];        int32_t
params[2];        buffer[0] = NET_COMMAND_PACKET;
```



```

switch (ch) {    case 'm':    case 'M': {    initscr();
cbreak(); // Initialize ncurses window    noecho();
timeout(300); // Check every 100ms (responsive yet safe)
curs_set(0); // Invisible cursor    int running = 1;    char
CommandSent = 'r'; // Track if forward command is active
while (running) {    int c = getch(); // Non-blocking check
every 100ms    if (c == 'w') {    flushinp();
if (CommandSent != 'w') { // Ignore repeated commands
buffer[1] = 'w';    params[0] = 0;    params[1]
= 0;    memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));    CommandSent =
'w';
    }    }    if (c == 's') {
flushinp();    if (CommandSent != 's') {
buffer[1] = 's';    params[0] = 0;
params[1] = 0;    memcpy(&buffer[2], params,
sizeof(params));    sendData(conn, buffer,
sizeof(buffer));    CommandSent = 's';
    }    } else if (c == 'a') {
flushinp();    if (CommandSent != 'a') {
params[0] = 0;    params[1] = 0;
buffer[1] = 'a';    memcpy(&buffer[2], params,
sizeof(params));    sendData(conn, buffer,
sizeof(buffer));

```



```

        CommandSent = 'a';
    } else if (c == 'd') {
flushinp();
        if (CommandSent != 'd') {
buffer[1] = 'd';
            params[0] = 0;
params[1] = 0;
            memcpy(&buffer[2], params,
sizeof(params));
            sendData(conn, buffer,
sizeof(buffer));
            CommandSent = 'd';
        } else if (c == ERR) {
flushinp();
// Key is released (no key detected)
            if (CommandSent !=
'r') {
buffer[1] = 'r'; // send stop command exactly
once
            params[0] = 0;
            params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
            CommandSent =
'r'; // Clear buffer to prevent flooding
        } else if (c == 'q' || c == 'Q') {
flushinp();
            running = 0;
            if
(CommandSent != 'r') {
// Ensure robot
stops if quitting
            buffer[1] = 'r';
params[0] = 0;
            params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = 'r';
        } else if (c == '=') { // Speed +=
5
flushinp();
            if (CommandSent !=
'=') {
buffer[1] = '=';
params[0] = 0;
            params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));

```



```

        sendData(conn, buffer, sizeof(buffer));
CommandSent = '=';
    }        } else if (c == '-') { // Speed -=
5        flushinp();        if (CommandSent != '-'
') {        buffer[1] = '-';        params[0]
= 0;        params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '-';
    }        } else if (c == '1') {
// Speed = 10

        flushinp();        // NOTE 2: flushinp() called every loop
if (CommandSent != '1') { // NOTE 1: ALGORITHM TO PREVENT SENDING
buffer[1] = '1';        REPEATED COMMANDS        params[0] =
0;        params[1] = 0;        memcpy(&buffer[2], params,
sizeof(params));        sendData(conn, buffer, sizeof(buffer));
CommandSent = '1';
    }        } else if (c == '2') { // Speed =
20        flushinp();        if (CommandSent !=
'2') {        buffer[1] = '2';
params[0] = 0;        params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '2';
    }        } else if (c == '3') { // Speed =
30        flushinp();        if (CommandSent !=
'3') {        buffer[1] = '3';
params[0] = 0;        params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));

```



```

        sendData(conn, buffer, sizeof(buffer));
CommandSent = '3';
    }        } else if (c == '4') { // Speed =
40        flushinp();        if (CommandSent !=
'4') {        buffer[1] = '4';
params[0] = 0;        params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '4';
    }        } else if (c == '5') { // Speed =
50        flushinp();        if (CommandSent !=
'5') {        buffer[1] = '5';
params[0] = 0;        params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '5';
    }        } else if (c == '6') { // Speed =
60        flushinp();        if (CommandSent !=
'6') {        buffer[1] = '6';
params[0] = 0;        params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '6';
    }        } else if (c == '7') { // Speed =
70        flushinp();        if (CommandSent !=
'7') {        buffer[1] = '7';
params[0] = 0;        params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));

```



```

        CommandSent = '7';
    }          } else if (c == '8') { // Speed =
80      flushinp();          if (CommandSent !=
'8') {          buffer[1] = '8';
params[0] = 0;          params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '8';
    }          } else if (c == '9') { // Speed =
90      flushinp();          if (CommandSent !=
'9') {          buffer[1] = '9';
params[0] = 0;          params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '9';
    }          } else if (c == '0') { // Speed =
100     flushinp();          if (CommandSent !=
'0') {          buffer[1] = '0';
params[0] = 0;          params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
CommandSent = '0';
    }
}
napms(300);      }
endwin();
break;      }
case 'K':
    getParams(params);
    buffer[1] = ch;

```

```

        memcpy(&buffer[2], params, sizeof(params));
sendData(conn, buffer, sizeof(buffer));
break;    case 'a':
        params[0] = 0;    params[1] = 0;
memcpy(&buffer[2], params, sizeof(params));
buffer[1] = ch;    sendData(conn, buffer,
sizeof(buffer));    break;    case 'Q':
quit = 1;    break;    default:
        printf("BAD COMMAND\n");    }
}    printf("Exiting keyboard
thread\n");    stopClient();
EXIT_THREAD(conn);    return NULL;
}

```

Appendix A, Listing 10: Colour Sensor Set-up

```

void setup_colour_sensor(){
cli();
    // Reset Timer 5
    TCCR5A = 0;
    TCCR5B = 0;
    DDRL |= (S1 | S2 | S0);    // setting PIN 47, 45, 43, 42 to output
    DDRG |= S3;    //white wire
    DDRL &= ~COL_FREQ;    // set ICP pin to input

    TCCR5B = (1 << ICNC5) | (1 << ICES5) | (1 << CS50);
    // Input capture on rising edge, no prescaler, noise cancellation
    ICR5 = 0; // Clear previous capture values
    TIMSK5 &= ~(1 << ICIE5); // Disable interrupt initially to avoid premature

```

```
triggers sei();  
}
```

Appendix A listing A-11: Colour Sensing Algorithm

```
void wait_for_period(){  
done = false; TCNT5 =  
0; rising_edge_count =  
0; period = 0;  
while(!done); }  
ISR(TIMERS5_CAPT_vect){ if (rising_edge_count == 0) {  
TCNT5 = 0; //reset count start of first period } else if  
(rising_edge_count >= 2 && rising_edge_count <= 11){ period  
+= ICR5; TCNT5 = 0;  
} if (rising_edge_count ==  
11){ done = true; }  
rising_edge_count++; }  
float detect_frequency(){ PORTL |= (S1 |  
S0); // sets it to 100% mode  
TIMSK5 |= (1 << ICIE5);  
wait_for_period(); TIMSK5 &= ~(1  
<< ICIE5); float avg_period =  
period / 11.0; return CLOCK_FREQ  
/ avg_period;
```

```

} void detect_colour(){
char numBuffer[16]; //
Enable red photodiode
    PORTL &= ~S2;    PORTG &= ~S3;    float red_freq =
detect_frequency();    dtostrf(red_freq, 6, 2, numBuffer); //
width=6, precision=2    dbprintf("Red value: %s", numBuffer);
    // Enable green photodiode
    PORTL |= S2;    PORTG |= S3;    float green_freq =
detect_frequency();    dtostrf(green_freq, 6, 2, numBuffer); //
width=6, precision=2    dbprintf("Green value: %s", numBuffer);
    // Enable blue photodiode    PORTL &= ~S2;
    dtostrf(detect_frequency(), 6, 2, numBuffer); // width=6, precision=2
    dbprintf("Blue value: %s", numBuffer);    if (red_freq > green_freq) {
sendMessage("Colour Detected: RED");    } else {    sendMessage("Colour
Detected: GREEN");    }
}

```

Appendix A, Listing A12: CLAW SETUP

```

void setup_claw() {
cli(); // Reset
Timer 5
    TCCR5A = 0;
    TCCR5B = 0;

    DDRL |= (1 << PL3) | (1 << PL4) | (1 << PL5); // OC5A/B/C outputs

```

```

ICR5 = 40000; // 20 ms period
OCR5A = 2000; // 1.5 ms pulse → center
OCR5B = 4000; // 1.0 ms pulse → left
OCR5C = 4000; // 2.0 ms pulse → right
    setup_claw_TCCR();    open_claw();
close_claw();    sei(); } void
setup_claw_TCCR(){ // Fast PWM,
Mode 14 (ICR5 as TOP)
    TCCR5A = (1 << COM5A1) | (1 << COM5B1) | (1 << COM5C1) | (1 << WGM51);
TCCR5B = (1 << WGM53) | (1 << WGM52) | (1 << CS51); // Prescaler = 8
}

```

Appendix A, Table 1: TPacket Codes & Types .

PACKET_TYPE	PACKET CODE
COMMAND	0
RESPONSE	1
ERROR	2
MESSAGE	3
HELLO	4

Appendix A, Table 2: Response Codes & Types

RESPONSE_TYPE	RESPONSE CODE
OK	0
STATUS	1
BAD_PACKET	2
BAD_CHECKSUM	3
BAD_COMMAND	4
BAD_RESPONSE	5

Appendix A, Table 3: Command Types & their corresponding codes and input keys within the client. Additionally other client commands are included.

INPUT KEY		COMMAND_TYPE	COMMAND CODE
"Normal Mode" w/ target speed & Distance	"f"	FORWARD	0
	"b"	REVERSE	1
	"l"	TURN_LEFT	2
	","	TURN_RIGHT	3
Normal Mode	"r"	STOP	4
ncurses Mode	ERR		
Normal Mode	"g"	GET_STATS	5
	"h"	CLEAR_STATS	6
	"c"	GET_COLOUR	7
	"p"	CLOSE_CLAW	8
	"o"	OPEN_CLAW	9
	"i"	DISPENSE_MEDPACK	10
ncurses Mode	"w"	KEEP_GOING_FORWARD	11
	"s"	KEEP_GOING_BACKWARD	12
	"a"	KEEP_GOING_LEFT	13
	"d"	KEEP_GOING_RIGHT	14
	"="	INCREASE_SPEED	15
	"_"	DECREASE_SPEED	16
	"1"	SET_SPEED_10	17
	"2"	SET_SPEED_20	18
	"3"	SET_SPEED_30	19
	"4"	SET_SPEED_40	20
	"5"	SET_SPEED_50	21
	"6"	SET_SPEED_60	22
	"7"	SET_SPEED_70	23
	"8"	SET_SPEED_80	24
	"9"	SET_SPEED_90	25
	"0"	SET_SPEED_100	26
Client Specific Commands	"m"	ENTER ncurses Mode	NA
	"q"	If in ncurses Mode: Exit ncurses Mode	NA
		If in normal mode: Exit Connection	NA

Appendix B – Photo Documentation of Alex

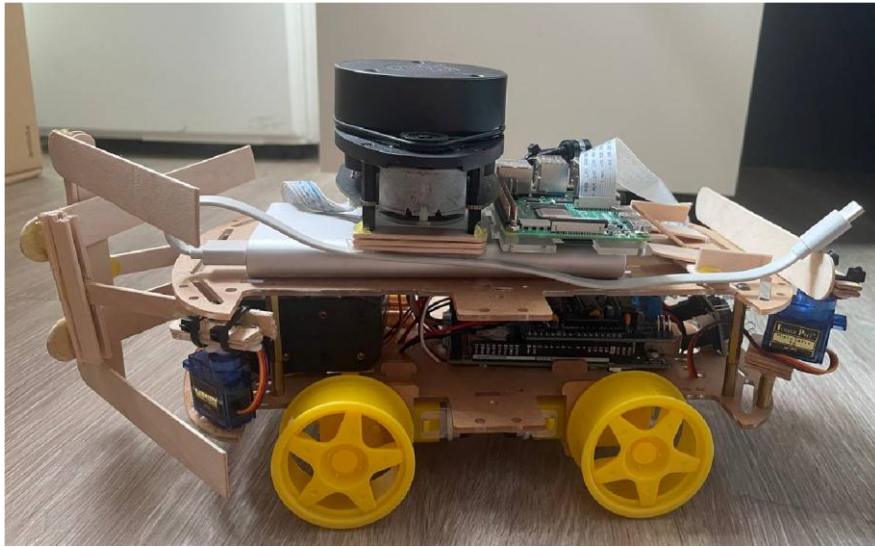


Figure B-1: Alex's Right Side

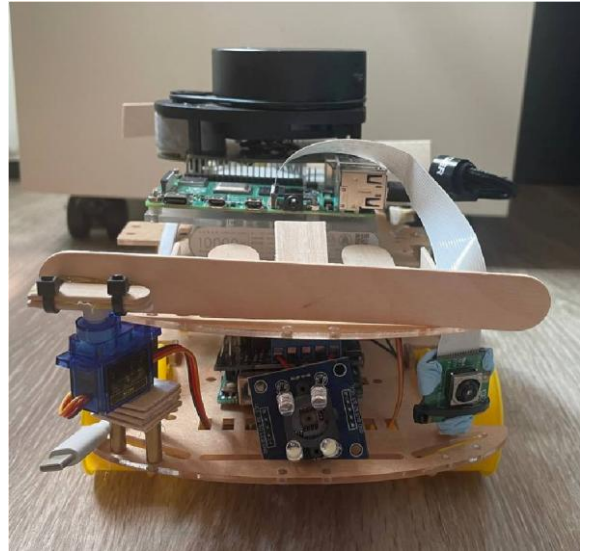


Figure B-2: Alex's Front Side

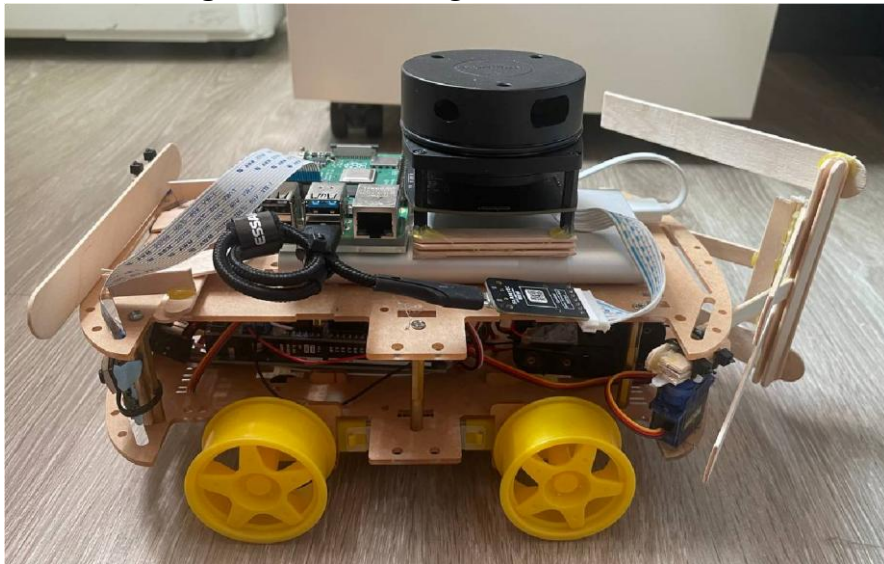


Figure B-3: Alex's Left Side

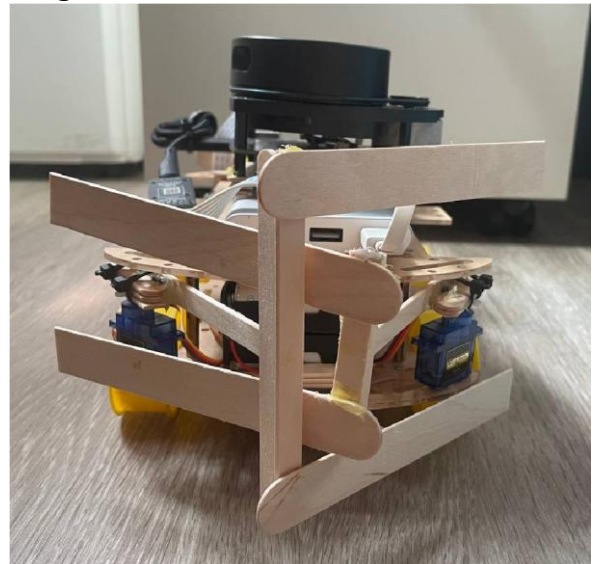


Figure B-4: Alex's Back side

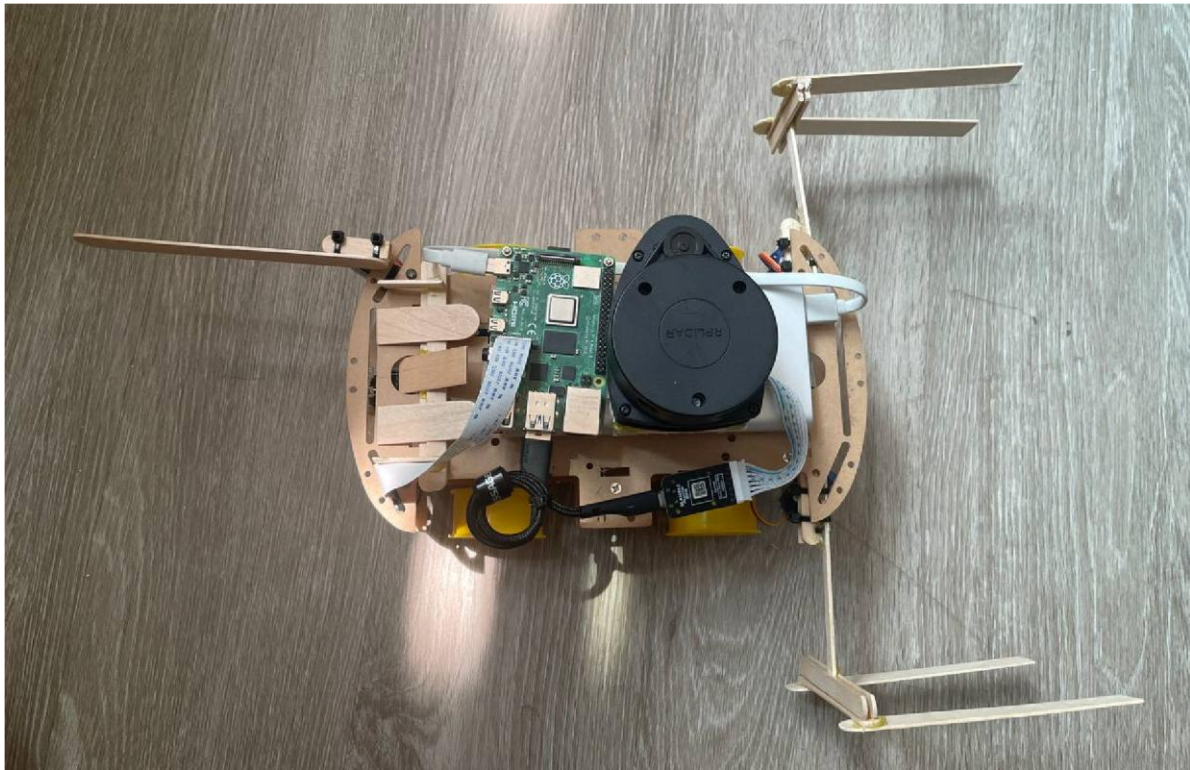


Figure B-5: Top view of Alex's top layer with extended dispenser and Claws

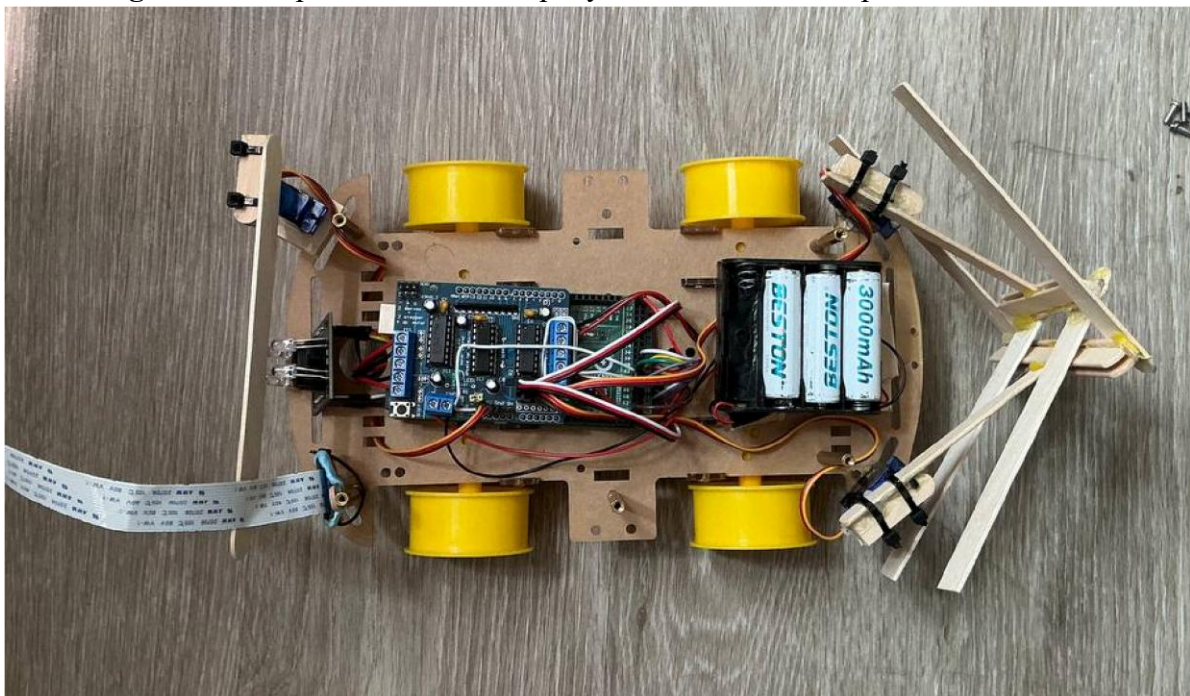


Figure B-6: Top view of Alex's base layer with closed dispenser and Claws

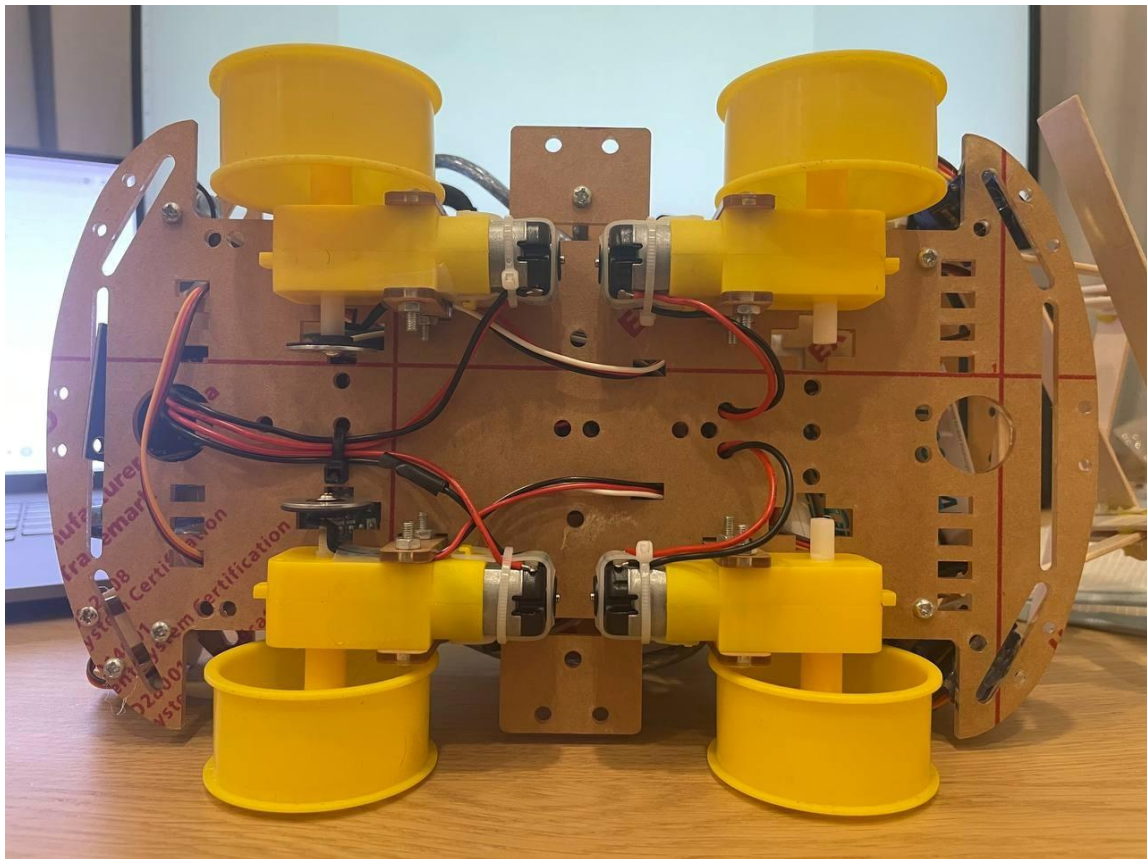


Figure B-7: Bottom view and wiring of Alex's

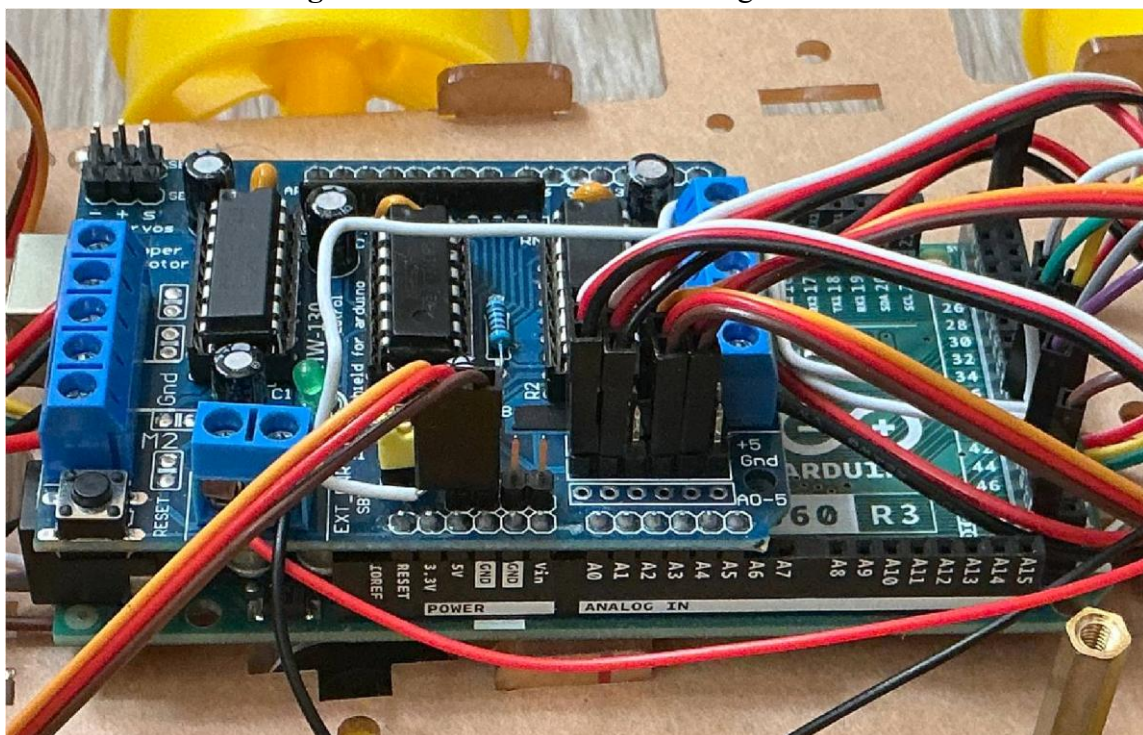


Figure B-8: Soldering and wiring on Alex's Motor shield & Arduino

Appendix C

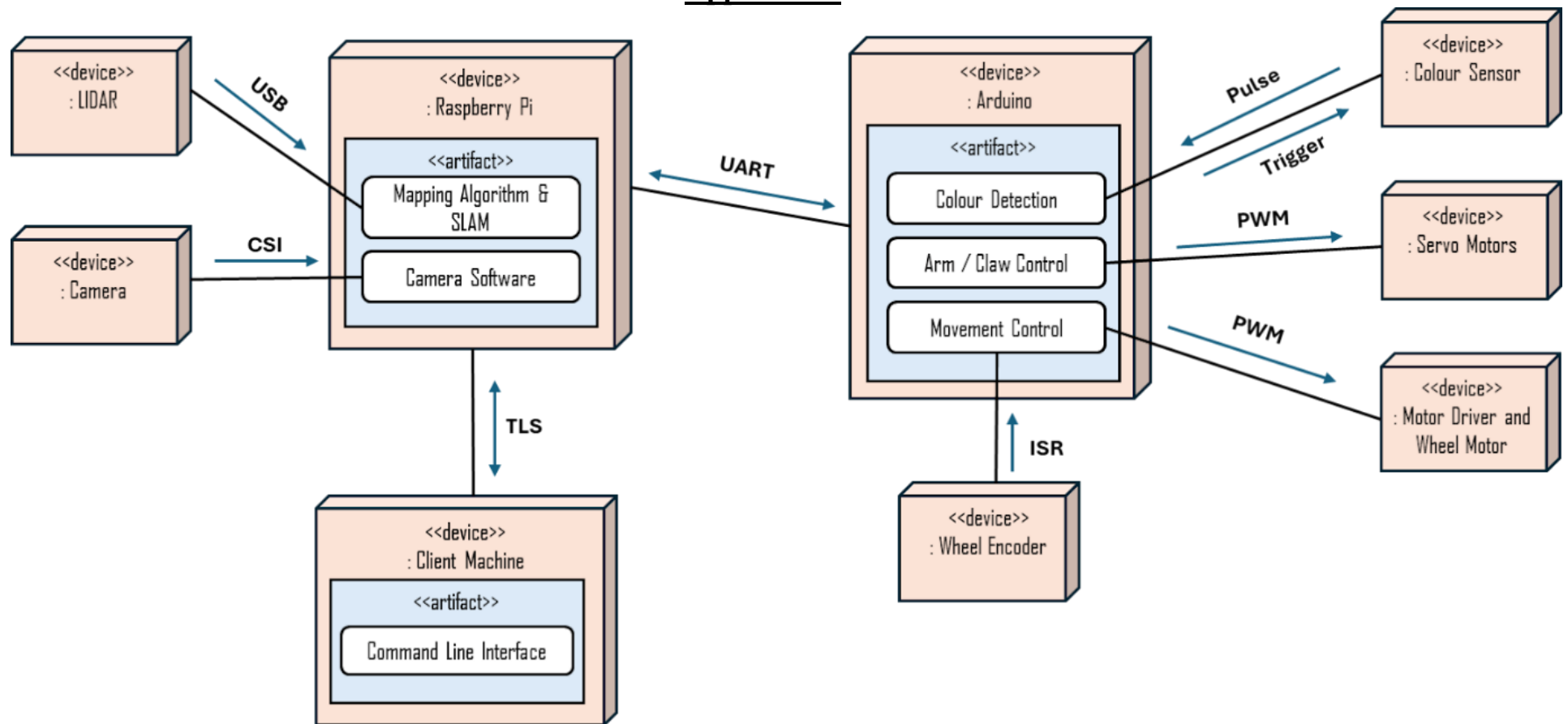


Figure C-1: Enlarged Diagram of our Hardware Decoration

Appendix D

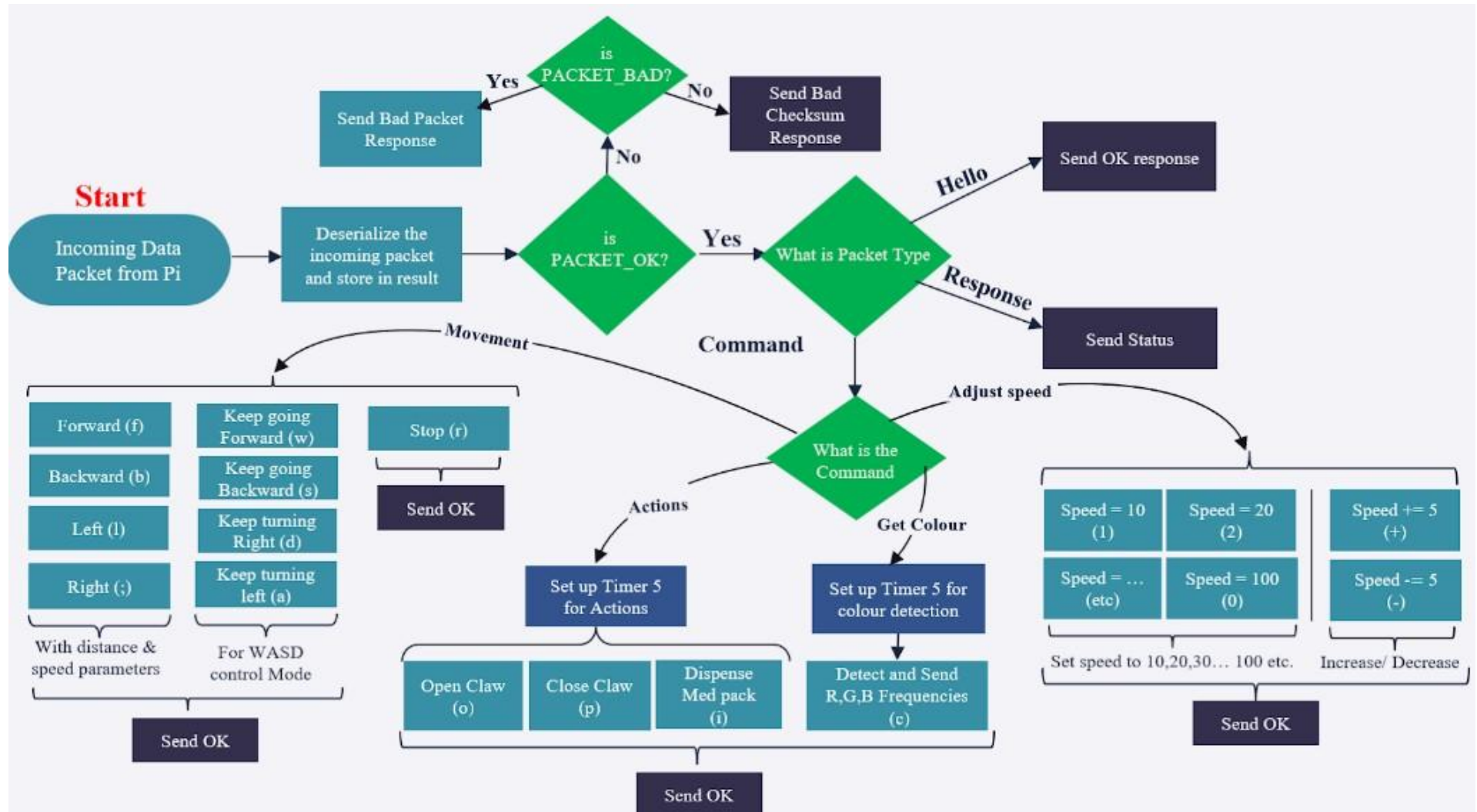


Figure D-1: Enlarged Diagram of our High Level Algorithm of our Arduino Mega

References

Klamt, T., Rodriguez, D., Schwarz, M., Lenz, C., Pavlichenko, D., Droschel, D., & Behnke, S. (2018). *Supervised autonomous locomotion and manipulation for disaster response with a centaur-like robot*. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). Available at: https://www.ais.uni-bonn.de/papers/IROS_2018_Klamt.pdf [Accessed 30th March 2025]

Microchip Technology Inc. (2014). *ATmega640/V-1280/V-1281/V-2560/V-2561/V datasheet* (Document DS40002211A). Personal copy provided by CG2111A at NUS. Available at: <https://www.microchip.com/en-us/product/ATmega2560> [Accessed 25th April 2025]

Shanghai Slamtec Co., Ltd. (2020). *RPLIDAR A1 low cost 360 degree laser range scanner: Introduction and datasheet (Model A1M8, Rev. 3.0)*. Shanghai Slamtec Co., Ltd. Personal copy provided by CG2111A at NUS.

Simon D. Levy. (n.d). *BreezySLAM*, GitHub repository, Available at: <https://github.com/simondlevy/BreezySLAM>, [Accessed 9th April 2025]

Texas Advanced Optoelectronic Solutions, Inc. (2009). *TCS3200, TCS3210 programmable color light-to-frequency converter* (TAOS099) [PDF]. Available at: <https://drive.google.com/file/d/0BzFWfMiqqjyqTEl6RHN4TGdTb2s/view?resourcekey=0-A8HXIH2cUmRAJMkoHLx6tQ> [Accessed 25th April 2025]

Texas Instruments. (2016). *L293, L293D quadruple half-H drivers* (SLRS008D, Rev. D). Personal copy provided by CG2111A at NUS. Available at: <https://www.ti.com/product/L293D> [Accessed 25th April 2025]

Zhou, Y., Yuan, Y., Zhao, R., & Wu, Z. (2022). *Earthshaker: A Mobile Rescue Robot with Autonomous Functions in a Search and Rescue Environment*. Journal of Unmanned System Technology and Control. Available at: <https://justc.ustc.edu.cn/article/doi/10.52396/JUSTC-2022-0066> [Accessed 30th March 2025]